Date: 31-03-2026
Roll No. B25110206030

Week 12 – Sprint Animation

Task: Focus on how nicely and cleverly the switch statement is used multiple times to achieve sprite animation. It mainly need two files (i) The available pixmap.h (ii) our own main.cpp.

Program:

lab12a.cpp U

lab12a > lab12a.cpp > pos_Y

```cpp
#include <GL/glut.h>
#include "pixMap.h"
#include <mmsystem.h>

#define IMAGES 5
RGBApixmap pix[IMAGES];

enum State { STANDING = 0, RUNNING1 = 1, RUNNING2 = 2, RUNNING3 = 3, JUMPING = 4, DIE };
enum ModeType { STAY, RUN, JUMP, DEAD };

State state = STANDING;
ModeType mode = STAY;

float pos_X = 0.0f;
float pos_Y = 0.0f;

void changePosition(float dx, float dy) {
    pos_X += dx;
    pos_Y += dy;
}

void setPosition(float x, float y) {
    pos_X = x;
    pos_Y = y;
}

void changeMode(ModeType m) {
    if (mode == m) return;

    switch(m) {
        case STAY: state = STANDING; break;
        case RUN:  state = RUNNING1; break;
        case JUMP: state = JUMPING; break;
    }
    mode = m;
}

void run() {

    switch (state) {
        case RUNNING1: state = RUNNING2; break;
        case RUNNING2: state = RUNNING3; break;
        case RUNNING3: state = RUNNING1; break;
        default: state = RUNNING1; break;
    }
```

lab12a.cpp U

lab12a > lab12a.cpp > changePosition(float, float)

```cpp
void render() {
    switch (mode) {
        case STAY:
            glRasterPos2f(pos_X, pos_Y);
            pix[0].mDraw();
            break;

        case RUN:
            run();
            glRasterPos2f(pos_X, pos_Y);
            pix[state].mDraw();
            break;

        case JUMP:
            glRasterPos2f(pos_X, pos_Y);
            pix[4].mDraw();
            break;
    }
}

void myInit(void) {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);

    glViewport(0, 0, 640, 480);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 640, 0, 480);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    pix[0].readBMPFile("images/MarioStanding.bmp", 1);
    pix[1].readBMPFile("images/MarioRun1.bmp", 1);
    pix[2].readBMPFile("images/MarioRun2.bmp", 1);
    pix[3].readBMPFile("images/MarioRun3.bmp", 1);
    pix[4].readBMPFile("images/MarioRun4.bmp", 1);

    for (int i = 0; i < IMAGES; i++) {
        pix[i].setChromaKey(192, 192, 192);
    }

    glEnable(GL_ALPHA_TEST);
    glAlphaFunc(GL_EQUAL, 1.0);

    changeMode(STAY);
```

lab12a.cpp U

lab12a > lab12a.cpp > pos_Y

```cpp
void myInit(void) {
    for (int i = 0; i < IMAGES; i++) {
        pix[i].setChromaKey(192, 192, 192);
    }

    glEnable(GL_ALPHA_TEST);
    glAlphaFunc(GL_EQUAL, 1.0);

    changeMode(STAY);

    PlaySound("sounds/backmusic.wav", NULL, SND_FILENAME | SND_ASYNC | SND_LOOP);
}

void display(void) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    render();

    glFlush();
}

void pressKeySpecial(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT:
            changePosition(-20, 0);
            changeMode(RUN);
            break;

        case GLUT_KEY_RIGHT:
            changePosition(20, 0);
            changeMode(RUN);
            break;

        case GLUT_KEY_UP:
            if (pos_Y < 50) {
                changePosition(0, 50);
                changeMode(JUMP);
            }
            break;
    }
    glutPostRedisplay();
}
```

lab12a.cpp U

lab12a > lab12a.cpp > changePosition(float, float)

```cpp

void releaseKeySpecial(int key, int x, int y) {
    switch (key) {
        case GLUT_KEY_LEFT:

            break;

        case GLUT_KEY_RIGHT:

        break;

        case GLUT_KEY_UP:
            if (pos_Y != 0) {
                setPosition(pos_X, 0);
                changeMode(STAY);
            }
            break;
    }
    glutPostRedisplay();
}

int main(int argc, char **argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(10, 10);
    glutCreateWindow("Sprite Animation Demo");

    myInit();

    glutDisplayFunc(display);
    glutSpecialFunc(pressKeySpecial);
    glutSpecialUpFunc(releaseKeySpecial);

    glutMainLoop();
    return 0;
}
```

Date: 31-03-2026
Roll No. B25110206030

Lab Manual:

Phase 1: Organizing Assets (Setting the Stage)
To make sure our code can find Mario's pictures and his background music, our project folder needs to be perfectly organized.

1. **The Main Folder:** Create a folder on your computer called MarioGame. Everything goes inside here!
2. **The images Folder:** Inside your main folder, create a new folder named images.
    - Put your 5 Mario pictures here (MarioStanding.bmp, MarioRun1.bmp, etc.).
    - *Important Rule:* They **must** be saved as 24-bit BMP files, otherwise our code won't know how to read them!
3. **The sounds Folder:** Create another folder named sounds.
    - Put your background music file here (backmusic.wav). It must be a .wav file, not an .mp3.
4. **The Helper File (pixMap.h):** Put this file directly into your main folder. Think of pixMap.h as our custom paintbrush—it is a special tool provided by the teacher that teaches our C++ code how to draw BMP images and make their backgrounds transparent.
5. **Your Code (main.cpp):** This is where you write your actual game code. Save it right next to pixMap.h.

Phase 2: How the Code Works (The Crew & Actors)

Once your files are organized, we write the code. Here is exactly what every function in our program does, in the order they work together:

1. changePosition(dx, dy)
- Makes Mario take a step. It adds numbers to his current spot to move him left, right, up, or down.

2. setPosition(x, y)
- Teleports Mario. It ignores where he is currently standing and instantly places him at an exact, specific coordinate.

3. changeMode(ModeType m)
- Switches Mario's "brain" to a new action. If he was standing, this tells him, "Okay, now you are running" or "Now you are jumping."

4. run()
- Flips the pages of the animation book. Every time it runs, it switches the picture from Running Frame 1, to Frame 2, to Frame 3, and loops back around so his legs keep moving.

5. render()
- The artist. It checks what Mario is supposed to be doing (standing, running, or jumping), grabs the correct picture, and draws it on the screen at his exact coordinates.

6. myInit()

- The backstage setup crew. Before the game starts, this sets up the camera lens, loads your 5 image files from the images folder into memory, makes their grey backgrounds invisible, and turns on the background music from the sounds folder.

7. display()

- The movie projector. It wipes the screen completely clean, asks render() to draw the newest picture of Mario, and then flashes that finished picture onto your monitor.

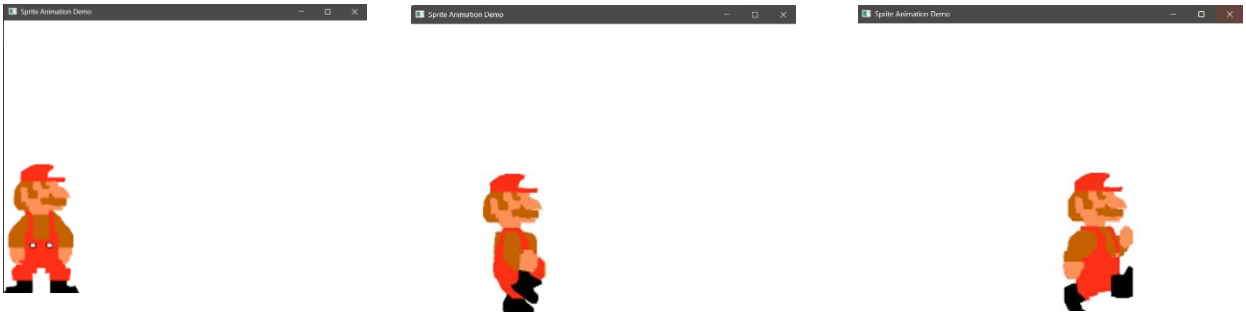
8. pressKeySpecial(...)

- Listens for when you push an arrow key *down*. If you press Right, it tells Mario to take a step right and change his mode to RUN.

9. releaseKeySpecial(...)

- Listens for when you *let go* of an arrow key. If you let go of the Up arrow, it brings Mario back to the ground and tells his mode to STAY.

10. main()

- The power button. It creates the actual window on your computer screen, calls the setup crew (myInit), connects your keyboard, and puts the game into an infinite loop so it stays open and running until you click the 'X'.

Output: